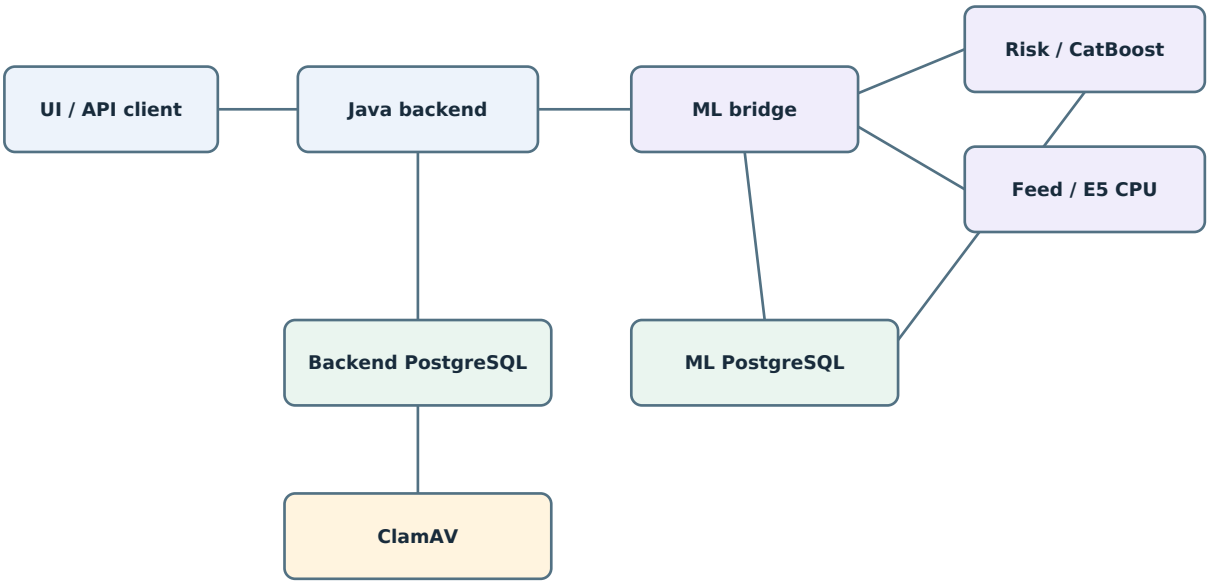
Backend + ML Интеграционная архитектура

22 октября 2026 · Runtime и границы поставки

Приоритет доработок ML. Java backend сохраняет бизнес-решения и пользовательское состояние. Проверки, ограничения и источники отделены от исторической спецификации.



Актуальность: 22 октября 2026. Этот документ описывает фактическую интеграционную поставку. Комплект HACKATHON-1.5 сохранён как исходная спецификация более широкого продукта; его исторические PDF и PASS не являются доказательством запуска этой версии.

Состав и источники

База ML: integration/ml-audit-20261021, commit af86831. Backend: backend/p15-ml-rfi-decisions, commit 4f229b0, включает P03/P05/P08/P25/P10. Полные SHA и состав веток - reports/integration-20261022/source-versions.json. Рабочая ветка: integration/backend-ml-mvp. Исходные рабочие папки пользователя не изменялись; интеграция создана в отдельном checkout.

Risk: неизменённый обученный CatBoost baseline ml/artifacts/baseline, восемь финансовых коэффициентов, BANKRUPTCY_PROXY / DEMONSTRATION / DEMO_RESEARCH. Feed: локальная E5 CPU и каталог v1 из 36 карточек. Числа и исходы скоринга синтетические; score не является банковской PD.

Более свежие ветки 802f998 (антифрод) и 87af049 (E5+CatBoost, новый каталог) изучены отдельно. Они не активируются в этой интеграции автоматически. Антифроду нужен доверенный независимый reference, которого текущий backend не предоставляет; подмена его заявленными финансами недопустима. Новый рекомендатель требует отдельного модельного релиза и оценки каталога, данных и качества. Код и evidence этих веток остаются сохранёнными в Git. См. аудит и backlog.

Архитектура и владельцы

Схема компонентов приведена на титульной странице и в отдельном SVG.

Backend владеет сессиями, контекстом компании, подтверждением финансов, неизменяемыми финансовыми снимками, RFI, решениями, предложениями, предпочтениями, выдачами и реакциями. ML не создаёт пользователей, не выдаёт права и не принимает кредитных решений. Backend и ML используют отдельные БД; ни один сервис не читает таблицы другого владельца напрямую.

В Compose публичен только backend на 127.0.0.1:8080. Risk, Feed, bridge, обе БД и ClamAV находятся в сети internal: true. Установка зависимостей и подготовка весов происходят до запуска изолированного runtime. Образы пока закреплены тегами; digest образов сохраняется в evidence Linux-прогона. Это не обещание воспроизводимости старого образа по изменяемому тегу.

Совместимость Risk

Backend продолжает использовать ML_BASE_URL, X-Service-Token и свои DTO. ML bridge переводит авторизацию в Bearer для Risk и направляет запросы в отдельные сервисы. Исходный native Risk API сохранён.

1. Backend передаёт точный financialSnapshotId, digest, версию и время создания неизменяемого снимка. Оно служит временем доступности подтверждённых входов и стабильным asOf попытки; это не утверждение о времени публикации внешней отчётности. 2. Bridge проверяет backend digest по сериализации P15: statementVersion, порядок LinkedHashMap, восемь значений и полные documentId/documentVersion/objectVersion. Денежные значения не округляются, не восстанавливаются и не исправляются. 3. Для native ML вычисляется отдельный canonical digest. Обе ссылки и преобразование сохраняются в журнале. Ответ backend содержит его исходную финансовую ссылку и отдельный проверяемый digest представления признаков. 4. Специальный ML input profile поддерживает годовую DEMO-форму и допуск баланса backend: max(100 копеек, 1% активов с HALF_UP). Все восемь коэффициентов рассчитываются по исходным значениям;

эквивалентность legacy transform проверена. Документы сохраняются в provenance, их текст не входит в модель. Native legacy профиль остаётся строгим и побайтно сохраняет исходный features.py. 5. Журнал ml.bridge_call фиксирует input hash, bundleDigest, featureCodeDigest и inputAdapterDigest до обращения к Risk. Повторы сериализуются блокировкой PostgreSQL, конфликт входа даёт 409. Успешный результат возвращается только после commit; после отказа сохранения native идемпотентность позволяет безопасно повторить операцию. 6. RFI использует новый финансовый снимок. Идентичность нового ML-профиля связана с финансовым снимком, поэтому исходный submission не блокирует обработку исправленной формы.

Risk использует исходную модель и служит единственным местом inference. Bridge не загружает CatBoost и не считает score. Изменение релиза не подменяет закреплённую модель незавершённой попытки; при недоступности закреплённого bundle возникает конфликт, backend отправляет дело на MANUAL. Для восстановления нужно вернуть прежний bundle или начать отдельную явную попытку рассмотрения, а не переписывать журнал.

Политика решений

Авторитет - Java DemoPolicyV1, версия demo-policy-v1. Порядок: неполные данные/противоречие выручки (>50%) - RFI; возраст бизнеса; нулевая выручка - MANUAL; отрицательный капитал или liabilities/assets >=0.9 - DECLINED; сбой Risk - MANUAL; лимит 25% выручки - ALTERNATIVE либо DECLINED ниже минимума; score >=0.5 - MANUAL; иначе APPROVED.

Порог 0.5 сохранён из backend для совместимости demo-policy. Он не равен историческому T_manual=0.03504223273828764 из исследовательской ML-policy и не сохраняет её долю ручной проверки. В MVP нет банковского обоснования ни одному из порогов. Это явное архитектурное решение для демонстрации, не калибровка и не перенос кредитной методологии.

ml/bridge/backend_policy.py - только приёмочный Python-эталон Java-policy. Старые scoring/demo_policy.py, S01-S06 и отчёты обучения остаются историческими сценариями своего профиля, а не источником решений интеграционного приложения. Контрактные двойники доступны для unit-тестов, но Compose принудительно включает ML_MODE=http.

Рекомендательная лента

Публичные endpoints: GET/PUT /api/v1/feed/preferences?companyId=..., GET /api/v1/feed, GET /api/v1/feed/saved, POST /api/v1/recommendation-feedback. Обязательны cookie, выбранная компания и X-Company-Context; мутации защищены CSRF. Состояние принадлежит (actor, company).

Backend фиксирует выдачу и исключает скрытые, подтверждённые VIEW и уже выданные в текущей подборке карточки. Cursor - случайный серверный идентификатор, связанный с пользователем, компанией, каталогом и версией предпочтений; повтор возвращает сохранённую порцию/exposure без нового ранжирования. Допускаются только исключение текущих скрытых карточек и обновление userReaction. Смена предпочтений инвалидирует старые cursor. Сохранённое полезное имеет отдельную savedVersion и не фильтруется обычными VIEW/issued.

Bridge принимает современный Feed DTO на /internal/v2/feed/rankings; старый RankingGateway поддержан отдельно на /internal/v2/rankings с переводом digest и allowlist enum-контекста. Свободный contextText в модель не передаётся. Источник applicationStage без явно выбранной заявки отсутствует и не выдумывается. PRODUCT-ранжирование моделью не объявляется реализованным: доступность продуктов определяет backend.

При отказе Feed модель заменяется существующими Python RulesRanker. Если недоступен весь bridge, backend использует эквивалентные Java-правила; порядок проверяется общими golden fixtures. Ошибки авторизации, каталога, DTO и некорректная перестановка не маскируются успехом. MODEL/RULES и releaseId видны в metadata.

Последняя серверно принятая HELPFUL/NOT_INTERESTED заменяет прежнюю реакцию; occurredAt не определяет приоритет. clientEventId долговечен и проверяет неизменность тела. Карточка должна присутствовать в выданном этому actor/company exposure. Выдача не создаёт VIEW: UI отправляет его после >=50% видимости в течение >=1000 мс; backend не может измерить реальную видимость браузера.

Минимальные изменения backend и их причины

- Передача createdAt финансового снимка: только backend знает достоверное время доступности подтверждённого входа. ML не может его придумать.
- Новый workspace Feed-срез и миграция V14: cookie/права, предпочтения, cursor, выдачи и реакции требуют backend-транзакций. Хранение этого в ML нарушило бы границы владельцев.
- Клиент современного Feed DTO и Java RulesRanker: публичная лента отсутствовала; сохранность выдачи и fallback при отказе всего ML-контура должны обеспечиваться backend.
- Изменения state machine заявок, решений, оферт и основной Java-policy не вводились. Остальные расхождения исправлены на стороне ML.

Запуск Docker Compose на Linux

1. python3 scripts/integration/configure.py создаёт приватный .env с новыми случайными значениями и не перезаписывает существующий. 2. docker compose config --quiet и docker compose build. 3. mkdir -p .local/feed-bundle. 4. На целевом Linux подготовить E5 с разрешённой сетью:

- docker run --rm --user "$(id -u):$(id -g)" -e HOME=/tmp
- -e HF_HUB_OFFLINE=0 -e TRANSFORMERS_OFFLINE=0
- -v "$PWD/.local/feed-bundle:/bundle" --entrypoint python
- credit-mvp-feed:local -m feed.prepare --bundle /bundle --download

5. docker compose up -d --wait --wait-timeout 300. 6. Установить jsonschema в отдельную Python-среду и запустить python scripts/integration/smoke.py --output reports/integration-linux/smoke.json.

Веса не лежат в Git. Подготовка закрепляет revision, SHA и библиотеки целевой среды. Mac bundle нельзя объявлять Linux bundle без пересборки manifest/проверки зависимостей. CPU-wheel PyTorch использует свою строку версии, отражённую в подготовленном manifest. ClamAV использует образ с предзагруженными сигнатурами и отключённым freshclam; обновление сигнатур - отдельная подготовка. См. официальную документацию ClamAV.

Для повторного автономного развёртывания нужны сохранённые образы (docker save/load), весь feed-bundle, .env и volumes. Первая установка требует сеть. Не выполнять down -v для обычного перезапуска или отката.

Проверка, выпуск и откат

Реальные проверки и точные ограничения находятся в reports/integration-20261022/. Linux Compose проверяется отдельным workflow .github/workflows/integrated-mvp.yml: чистая сборка, модельная подготовка, public HTTP/OpenAPI, restart, отказ Feed, отказ Risk и отсутствие внешнего TCP. Отсутствие успешного run означает NOT_VERIFIED_LINUX, независимо от локальных PASS.

Подробный аудит: docs/integration-audit.md. Регистрация источников, wire API bridge, public OpenAPI и новый архитектурный guide являются источниками поставки. scripts/integration/contracts.py пересобирает runtime-схемы и копию каталога backend; scripts/integration/build_docs.py пересобирает PDF, диаграмму и контрольные суммы интеграционных документов.

Откат: сохранить .env, feed-bundle и резервные копии обеих БД; вернуть прежние проверенные образы и модельный bundle. V14 аддитивна; её таблицы не удаляются при откате. Старую ML-поставку запускать на отдельном профиле; не направлять новые backend-запросы на прежний несовместимый API. После смены bundle незавершённые попытки завершаются старым релизом либо явно передаются на MANUAL.

Границы готовности

Публичный UI из соседнего проекта пока использует provisional-v1; для фактического backend поставлен отдельный типизированный adapter frontend/integration/api.ts. Полный UI ещё не принят. Антифрод, новый CatBoost-рекомендатель, OCR-worker, ассистент, принятие оферты и банковское оформление не объявляются частью проверенного сквозного сценария. Существующие upload/quarantine/AV и точные версии документов сохранены; в native HTTP-проверке путь ручного ввода не требует документов. Отказ сканера не даёт CLEAN.

Для банковского пилота нужны реальные зрелые labels, независимая валидация, согласованная policy, эксплуатационные SLO, управление секретами и инфраструктурная приёмка. Синтетические проверки подтверждают реализацию и сценарии, а не кредитное качество.

Аудит расхождений

22 октября 2026. Метод: история Git и source diff, чтение Java/Python, проверка native/runtime OpenAPI, тесты с фактическими HTTP и PostgreSQL. Изолированная рабочая копия сохраняет незакоммиченные материалы в исходных папках.

Реестр

ID	Подтверждённое расхождение	Риск	Решение и владелец	Статус
I01	P25 не включает P10/P15; P15 впереди P25 на два commit	Потеря готовых функций	Включить P15 по истории зависимостей	Реализовано
I02	HttpMlClient отправляет X-Service-Token на общий URL; Risk требует Bearer	Все реальные Risk-вызовы откажут	ML bridge переводит auth и маршрутизирует сервисы	Проверено HTTP
I03	Нет latestSourceAvailableAt и bundleDigest в Java-запросах	Неполный контракт/плавающий релиз	Backend передаёт immutable createdAt; ML journal закрепляет manifest	Проверено HTTP/PG
I04	Backend hash включает statementVersion и LinkedHashMap; ML canonical hash другой	Несовпадение snapshot	Проверять исходный hash; сохранять оба представления и digest	Проверено Java и Python
I05	Worker asOf меняется на повторе	409 на неизменном входе	Стабильное время финансового снимка из backend	Реализовано
I06	Native feature identity по submission мешает RFI с новой формой	Нельзя пересчитать исправление	Backend input profile связывает identity с financialSnapshotId	Проверено live Java/ML RFI
I07	Backend допускает balance tolerance и документы; legacy ML только точный баланс/без документов	Подтверждённая форма не проходит ML	Отдельный versioned ML input adapter, исходные суммы без исправления	Проверено tolerance и parity
I08	Java policy .5/50%/cap-first отличается от Python .035/20%/risk-first	Разные решения для одной заявки	Java-policy авторитетна; Python acceptance reference отдельно от обучения	Явно зафиксировано

ID	Подтверждённое расхождение	Риск	Решение и владелец	Статус
I09	Legacy RankingGateway hash - строки с переносом; Feed - sorted canonical JSON	409/некорректный подбор	ML legacy adapter + современный Feed endpoint	Проверено
I10	Публичная лента и её состояние отсутствовали	Готовая модель недоступна пользователю	Минимальный workspace Feed-срез с PG и изоляцией actor/company	Проверено public HTTP
I11	По умолчанию contract-double	Ложное ощущение готовности модели	Compose принудительно ML_MODE=http; evidence различает MODEL/RULES	Проверено Java
I12	Проверенный launcher macOS, нет общего Linux runtime	Невоспроизводима я передача	Compose, контейнеры, offline model preparation и Linux CI	Фактический Linux run указан в отчёте
I13	Frontend provisional-v1 отличается путями, полями, статусами и CAS	Нельзя подключить заменой URL	Поставлен типизированный клиент фактического API; полный UI отдельно	Adapter реализован, UI не принят
I14	Свежий антифрод требует независимого reference	Придуманное прои схождение/ложны й антифрод	Не активировать; требуется авторитетный источник и отдельная policy	Отложено
I15	Свежий Feed меняет каталог и добавляет CatBoost	Незаметная смена модели и критериев	Отдельный модельный release, без замены проверенного E5 baseline	Отложено

Критичные блокеры связности I01-I05/I09/I10/I11 закрываются в текущем срезе. I08 - сознательное демонстрационное решение, не банковская калибровка. I12 считается закрытым только после успешного Linux workflow; локальные native проверки не заменяют его. I13 не означает готовность существующих экранов.

Сравнение вариантов

Вариант	Изменения backend	Цена/ограничение	Решение
Переписать Java DTO и клиентов под каждый native ML	Существенные	Backend начинает зависеть от каждой версии ML; потребуется согласованная переделка	Не выбран
ML compatibility facade + отдельные Risk/Feed	Настройка, provenance, новый публичный Feed	Один дополнительный процесс и долговечный journal	Выбран: сохраняет backend и native API
Перенести Python inference внутрь Java	Существенные	Повторная упаковка моделей и валидация parity	Не выбран

Дополнительный процесс не получает бизнес-полномочий. Он выполняет техническое согласование, не создаёт независимую state machine заявки и не читает backend DB.

Дописать / изменить / перенести

- **ML:** bridge, журнал, version pinning, входной backend-profile, DTO/auth/digest, резервный подбор, контейнеры и тесты.
- **Backend:** созданное время financial snapshot в ML-входе; публичный Feed и его долговечное пользовательское состояние; fallback при недоступности всей ML-границы.
- **Сохранено:** Java-policy, процесс заявки/RFI, native scorer и обученный baseline, E5/каталог v1, legacy evidence.
- **Перенесено:** антифрод без независимого reference, новый CatBoost Feed без отдельной release-приёмки, полный frontend, OCR/RAG и банковское оформление.

Следующий backlog и оценка

Оценки ниже - диапазоны инженерных часов на последующую работу, а не журнал фактически затраченного времени; не включают ожидание данных и внешних согласований.

Приоритет	Работа	Зависимость	Оценка
P0 при отсутствии PASS	Завершить чистый Linux Compose run и устранить платформенные расхождения	Docker/Actions	4-12 ч
P1	Подключить существующие экраны к реальному API, статусы/CAS и browser e2e	frontend-команда, согласованный UX	12-24 ч

Приоритет	Работа	Зависимость	Оценка
P1	Расширить document reference сценарии и latency под несколькими клиентами	модельные данные, сканер	4-8 ч
P1	Проверить новый E5+CatBoost Feed и обновлённый каталог как отдельный release	фиксация dataset/protocol/gates	8-16 ч
P2	Подключить антифрод с настоящим независимым reference	источник/права/provenance, policy	8-20 ч после появления источника
P2	Закрепить образы digest, релизный архив, backup/restore и метрики нагрузки	выбранная платформа	6-12 ч

Даты внедрения следует назначать после закрытия P0 и согласования объёма UI/модельных релизов. Не обещаем банковский пилот на основании синтетической приёмки.